

Python - Notes

- `print ("_____")` : `print (var)`
 ↓ Statement ↓ value of variable.
- `age = 20` → int
`price = 19.95` → float
`first_name = "____"` → String
`Statement = True / False` → Boolean } Variables
- `name = input ("name? ")` → String type
 ↑ assignment
- - `int ( name )` → converts to int type.
 - `float (          )` → Real num
 - `bool (          )` → T/F
 - `str (          )` → String
- `print (" " + var)`
 this should be a string to add & print
- `course = 'Python for Beginners'`
`print ( course . _____ )`
 ↳ capabilities like (find , replace , upper , lower)
`print ( 'Python' in course )` → Boolean answer
 → prints true.
- Arithmetic operators:-
`print ( 10 + / * / - 3 )`
 Normal
`print ( 10 / 3 )` → prints floating → 3.333...
`print ( 10 // 3 )` → int type → 3
`print ( 10 % 3 )` → modulo
`print ( 10 ** 3 )` → prints → 1000 (10^3)

- Comparison operators ($>$, $<$, $>=$, $!=$, $==$)
 $x = 3 > 2 \rightarrow x = \text{True}$

- Logical operators; (and, or, not)
`print (price > 40 or price < 30)`

- `temp = 35`

| Use # for comments
|

Indentation $\rightarrow$

```
if temp > 30:
    print('Hot day')
elif temp > 20:
    print('Nice day')
else:
    print('Its cold')
```

|
|
|
|

Output :-

```
**
**
***
****
*****
```

- While loops :-

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

```
i = 1
while i <= 5:
    print(i * '*')
    i += 1
```

This is correct;
This will be read as '*' i times repeated

- Lists :-

→ type list

```
names = ['John', 'Ronn', 'Bonn', 'Sam', 'Mary']
```

```
names[0] = 'Jon'
```

```
print(names) → [ same as above ]
```

```
print(names[1]) → Jon
```

```
print(names[-2]) → Sam
```

```
print(names[0:3]) → ['Jon', 'Ronn', 'Bonn']
```

- list and tuples

```
x = [4, 'Hi', True]
```

```
x.append('Hello')
```

```
x.extend([4, 5, 5, 5])
```

```
print(x.pop())
```

→ prints True &
removes True

```
print(x.pop(0))
```

→ prints 4
and removes 4
from x.

↑ indices
↓

- sliced = [start : stop : step]

```
x = [0, 1, 2, 3, 4, 5, 6, 7, 8]
```

```
y = ['hi', 'hello', 'good', 'cya', 'sure']
```

```
s = "hello"
```

```
sliced = x[0:4:2]
```

```
print(sliced) → [0, 2]
```

```
sliced = x[4:2:-1]
```

```
print(sliced) → [4, 3]
```

```
sliced = x[::-1] → reversed x
```

```
sliced = s[::-1] → olleh
```

- List Methods:-

```
numbers = [1, 2, 3, 4, 5]
print(number.append(6)) → prints [1, 2, 3, 4, 5, 6]
print(numbers.insert(0, -1))
      ↳ prints [-1, 1, 2, 3, 4, 5]
prints (numbers.clear())
- "————— • remove(3)"
print(1 in numbers) → True
print(len(numbers)) → 5
```

- loops :-

```

numbers = [1, 2, 3, 4, 5]
for item in numbers:
    print(item)

```

Here it is iterator.

output:-

```

1
2
3
4
5

```

```

i = 0
while i < len(numbers):
    print(numbers[i])
    i += 1

```

same.

- Range functⁿs:-

numbers = range(5)
print(numbers) → prints 'range(0,5)'

```
for number in numbers: } prints:-
    print(number)
```

numbers = range(5,10) } prints:-
5
6
7
8
9

numbers = range(5, 10, 2) → prints 5, 7, 9

Annotations: 5 is 'start', 10 is 'stop', 2 is 'step'.

numbers = range(10, -1, -1)

- Tuples:- Can't be changed

$$\text{numbers} = (1, 2, 3, 3)$$

numbers[0] = 10 $\longrightarrow$ 'error'

numbers.count(3) $\longrightarrow$ output:- 2

numbers : index (3) $\longrightarrow$ output : 2 $\rightarrow$ first occurrence

- `x = set()`  Here, it stores data only one time. Used to see whether data exists or not

$$x = \text{set}()$$
$$S = \{4, 32, 2, 2\}$$

print(s) $\longrightarrow$ output :- {32,2,4}

S. add(5)

`print(s)` $\longrightarrow$ output : - {32, 2, 4, 5}

- Dictionaries $x = \{ \text{'key'} : \text{value} \}$

$$x = \{ \text{'key'} : 4 \}$$

↓ list, single value

`print (x['key'])` → prints: 4

$x[\text{'key 2'}] = 5$

$$x[2] = [2, 2, 1, 1]$$

print(k) $\longrightarrow$ output :-

$$\{ \text{'key'}: 4, \text{'key2'}: 5, 2: [2, 2, 1, 1] \}$$

- Comprehensions:-

$x = [x \text{ for } x \text{ in range}(5)]$
 $\text{print}(x) \longrightarrow [0, 1, 2, 3, 4]$

$x = [x+5 \text{ for } x \text{ in range}(5)]$
 $\text{print}(x) \longrightarrow [5, 6, 7, 8, 9]$

$x = [[0 \text{ for } x \text{ in range}(100)] \text{ for } x \text{ in range}(5)]$
 $\underbrace{\hspace{10em}}_{\text{lists with 100 '0's'}}$
 $\underbrace{\hspace{15em}}_{\text{5 lists with 100 '0's'}}$

lists $\rightarrow x = [i \text{ for } i \text{ in range}(100) \text{ if } i \% 5 == 0]$

sets $\rightarrow x = \{ \hspace{10em} \}$

dicts $\rightarrow x = \{ i : 0 \text{ for } i \text{ in range}(100) \text{ if } i \% 5 == 0 \}$

